

Fine-Tuning Eficiente de LLaMA 3.2 (1B) para Español Latinoamericano

con QLoRA (4-bit) y Unsloth en Infraestructura HPC-UCR: Informe Técnico Reproducible

Alison Lobo Salas
Universidad de Costa Rica (UCR), CIOdD
alison.lobo@ucr.ac.cr

MSI. Agustín Gómez Meléndez
CIOdD-UCR
agustin.gomez@ucr.ac.cr

Enero 2025

Abstract

Este informe técnico describe un flujo reproducible para adaptar el modelo de lenguaje grande `meta-llama/Llama-3.2-1B-Instruct` al español latinoamericano mediante *fine-tuning* eficiente con QLoRA (cuantización a 4 bits) y adaptadores LoRA, implementado con Unsloth y ejecutado en la infraestructura de computación de alto desempeño HPC-UCR (NVIDIA A100 80GB). El entrenamiento se realizó durante 60 épocas con longitud máxima de secuencia de 4096 tokens, optimizador AdamW y *scheduler* de *warmup* con decaimiento cosenoidal. Se reportan métricas agregadas de entrenamiento y validación junto con detalles completos de ejecución mediante SLURM para facilitar la replicación del proceso. Este trabajo forma parte de una colaboración más amplia con el proyecto Latam-GPT de CENIA (Chile) y contribuye al desarrollo de capacidades regionales en modelos de lenguaje contextualizados para América Latina.

Aportes y alcance

- Flujo completo y reproducible para ajuste fino eficiente mediante cuantización a 4 bits (QLoRA) y adaptadores de bajo rango (LoRA) en infraestructura académica. [1, 2]
- Configuración centralizada mediante archivo `config.py` y ejecución reproducible a través de sistema de colas SLURM para entornos HPC.
- Demostración de viabilidad técnica para entrenar modelos instructivos en español dentro de recursos computacionales universitarios limitados.
- Resultados cuantitativos presentados con tablas detalladas y análisis de convergencia, sin uso de gráficos.
- Documentación completa del proceso incluyendo manejo de errores, checkpoints automáticos y reanudación de entrenamiento.
- Contribución al proyecto Latam-GPT mediante validación de metodologías de entrenamiento y evaluación de modelos en contexto costarricense.

Contents

1	Introducción	3
1.1	Motivación y contexto	3
1.2	Objetivos del trabajo	3
1.3	Contribuciones principales	4
2	Fundamentos técnicos	4
2.1	Modelos de lenguaje y función objetivo	4
2.2	LoRA: adaptadores de bajo rango	4
2.3	QLoRA: cuantización y entrenamiento eficiente	5
2.4	Perplexity como métrica de evaluación	5
2.5	Optimización con AdamW	5
2.6	Scheduler de learning rate	6
3	Datos	6
3.1	Formato y estructura	6
3.2	Características del dataset	7
3.3	Procesamiento y tokenización	7
3.4	Partición train/validation	7
4	Infraestructura y entorno computacional	7
4.1	Hardware: HPC-UCR	7
4.2	Software y dependencias	8
4.3	Gestión de trabajos con SLURM	8
4.4	Configuración de paths y entorno	9
5	Metodología y configuración de entrenamiento	9
5.1	Modelo base	9
5.2	Hiperparámetros principales	10
5.3	Justificación de hiperparámetros	10
5.4	Estrategia de checkpointing	11
6	Reproducibilidad	11
6.1	Semillas aleatorias y determinismo	11
6.2	Entorno de dependencias	12
6.3	Contenedorización (opcional)	12
6.4	Instrucciones de replicación	12
7	Resultados y análisis	13
7.1	Métricas finales y convergencia	13
7.2	Evolución por época	13
7.3	Comportamiento del learning rate	14
7.4	Top-2 mejoras más significativas	14
7.5	Costos computacionales	14
7.6	Interpretación de resultados	14

8	Discusión	15
8.1	Validación de la metodología	15
8.2	Comparación con literatura	15
8.3	Contexto del proyecto Latam-GPT	16
8.4	Lecciones aprendidas	16
9	Limitaciones y trabajo futuro	17
9.1	Limitaciones actuales	17
9.2	Direcciones futuras	17
9.3	Recomendaciones metodológicas	18
10	Conclusiones	19
A	Comandos de uso	20
A.1	Instalación	20
A.2	Entrenamiento local	20
A.3	Entrenamiento en HPC con SLURM	21
A.4	Inferencia	21
B	Estructura de archivos del repositorio	21

1 Introducción

1.1 Motivación y contexto

En América Latina, la adaptación de modelos de lenguaje grande (LLMs) se ve limitada por tres factores principales: los altos costos computacionales asociados al entrenamiento de modelos desde cero, la predominancia del inglés en los datasets de pre-entrenamiento, y el acceso restringido a infraestructura de cómputo de alto desempeño. Estos factores generan una brecha tecnológica que dificulta el desarrollo de modelos contextualizados para las particularidades lingüísticas, culturales y de uso del español latinoamericano.

El proyecto Latam-GPT, liderado por el Centro Nacional de Inteligencia Artificial (CENIA) de Chile, busca desarrollar un modelo fundacional de lenguaje de código abierto específicamente entrenado para América Latina. Este proyecto representa un esfuerzo colaborativo regional para promover la soberanía tecnológica y lingüística mediante el desarrollo de capacidades locales en inteligencia artificial.

1.2 Objetivos del trabajo

Este trabajo presenta una estrategia eficiente y reproducible para adaptar el modelo `meta-llama/Llama-3.2-1B-Instruct` al español latinoamericano utilizando técnicas de *fine-tuning* eficiente. Los objetivos específicos son:

1. Demostrar la viabilidad técnica y operativa de entrenar modelos instructivos en español dentro de infraestructura académica (HPC-UCR).
2. Implementar y documentar un flujo completo de entrenamiento utilizando QLoRA (cuantización de 4 bits) que reduzca significativamente los requisitos de memoria GPU.
3. Evaluar el rendimiento del modelo mediante métricas intrínsecas (*loss*, perplejidad) durante el proceso de entrenamiento.

4. Proporcionar un marco reproducible que pueda ser adaptado por otras instituciones académicas de la región con recursos computacionales similares.
5. Contribuir a la validación de metodologías para el proyecto Latam-GPT mediante la documentación detallada de experiencias técnicas.

1.3 Contribuciones principales

Las contribuciones de este trabajo incluyen:

- Un pipeline completo de entrenamiento optimizado para GPUs A100 con gestión automática de checkpoints y reanudación.
- Configuración centralizada que facilita la experimentación con diferentes hiperparámetros sin modificar código fuente.
- Documentación exhaustiva del proceso de instalación, configuración y ejecución en entorno HPC con SLURM.
- Análisis cuantitativo detallado de la convergencia del modelo durante 60 épocas de entrenamiento.
- Scripts de inferencia que permiten evaluar cualitativamente el modelo entrenado.
- Código fuente abierto disponible en repositorio público para facilitar replicación y adaptación.

2 Fundamentos técnicos

2.1 Modelos de lenguaje y función objetivo

Los modelos de lenguaje grande basados en la arquitectura Transformer aprenden a predecir la probabilidad de secuencias de tokens mediante entrenamiento autosupervisado. Para un modelo instructivo, el objetivo es adaptar el modelo base para seguir instrucciones específicas dadas en lenguaje natural.

Sea un dataset de entrenamiento compuesto por pares de instrucción-respuesta $\{(x_i, y_i)\}_{i=1}^N$, donde x_i representa la instrucción (potencialmente con contexto adicional) y y_i la respuesta esperada. La función de pérdida de entropía cruzada token a token se define como:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log p_{\theta}(y_{i,t} \mid x_i, y_{i,<t}), \quad (1)$$

donde θ representa los parámetros del modelo, T_i es la longitud de la secuencia de salida i , y $p_{\theta}(\cdot)$ es la distribución de probabilidad modelada por la red neuronal.

2.2 LoRA: adaptadores de bajo rango

LoRA (*Low-Rank Adaptation*) [2] es una técnica de *fine-tuning* eficiente que mantiene congelados los pesos pre-entrenados del modelo base y entrena solo un conjunto pequeño de matrices de bajo rango que se añaden a las transformaciones lineales existentes.

Para una matriz de pesos $W \in \mathbb{R}^{d \times k}$ en el modelo original, LoRA añade una actualización de bajo rango:

$$W' = W + \Delta W = W + BA, \quad (2)$$

donde $B \in \mathbb{R}^{d \times r}$ y $A \in \mathbb{R}^{r \times k}$ con $r \ll \min(d, k)$. El rango r controla la capacidad expresiva de la adaptación. Durante el entrenamiento, solo se actualizan las matrices B y A , reduciendo drásticamente el número de parámetros entrenables.

El escalado de LoRA se controla mediante un factor α tal que la actualización efectiva es:

$$W' = W + \frac{\alpha}{r} BA. \quad (3)$$

Este factor permite controlar la magnitud de las actualizaciones independientemente del rango elegido.

2.3 QLoRA: cuantización y entrenamiento eficiente

QLoRA (*Quantized Low-Rank Adaptation*) [1] combina LoRA con cuantización de 4 bits del modelo base, reduciendo significativamente el uso de memoria GPU mientras mantiene la calidad del entrenamiento. La técnica utiliza:

- **Cuantización NF4** (*Normal Float 4*): Esquema de cuantización optimizado para pesos que siguen distribuciones normales, típicas en redes neuronales pre-entrenadas.
- **Doble cuantización**: Cuantización adicional de las constantes de cuantización para ahorrar más memoria.
- **Paginación optimizada**: Gestión eficiente de memoria GPU mediante offloading a CPU cuando es necesario.

La reducción de memoria permite entrenar modelos más grandes en GPUs con memoria limitada, o utilizar tamaños de batch más grandes en GPUs potentes como la A100.

2.4 Perplexity como métrica de evaluación

La perplexidad (*perplexity*) es una métrica estándar para evaluar modelos de lenguaje. Se define como la exponencial de la pérdida de entropía cruzada:

$$\text{PPL} = \exp(\mathcal{L}_{\text{eval}}). \quad (4)$$

Una perplexidad baja indica que el modelo asigna alta probabilidad a las secuencias correctas. Intuitivamente, puede interpretarse como el número promedio de opciones igualmente probables que el modelo considera en cada paso de predicción. Una reducción de perplexidad de 21.74 a 5.47 (como se observa en este trabajo) indica que el modelo ha mejorado significativamente su capacidad de modelar la distribución del lenguaje objetivo.

2.5 Optimización con AdamW

El optimizador AdamW [3, 4] es una variante mejorada de Adam que implementa decaimiento de pesos (*weight decay*) de forma correcta, separada de la actualización de gradientes. Esto mejora la regularización y generalización del modelo.

La actualización de parámetros en AdamW sigue:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (5)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (6)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad (7)$$

$$\hat{v}_t = v_t / (1 - \beta_2^t), \quad (8)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right), \quad (9)$$

donde η es la tasa de aprendizaje, λ es el coeficiente de decaimiento de pesos, y β_1, β_2 son los momentos del primer y segundo orden.

2.6 Scheduler de learning rate

El *scheduler* de tasa de aprendizaje implementado combina dos fases:

1. **Warmup lineal:** La tasa de aprendizaje aumenta linealmente desde 0 hasta el valor objetivo durante los primeros pasos. Esto estabiliza el entrenamiento inicial evitando actualizaciones muy grandes antes de que el optimizador acumule estadísticas de momento.
2. **Decaimiento cosenoidal:** Después del warmup, la tasa de aprendizaje decae siguiendo un coseno hasta aproximarse a cero al final del entrenamiento. Esto permite exploración inicial con pasos grandes y refinamiento progresivo con pasos pequeños.

El schedule matemático es:

$$\eta(t) = \begin{cases} \eta_{\max} \cdot \frac{t}{t_{\text{warmup}}} & \text{si } t \leq t_{\text{warmup}} \\ \eta_{\max} \cdot \frac{1}{2} \left(1 + \cos \left(\pi \frac{t - t_{\text{warmup}}}{T - t_{\text{warmup}}} \right) \right) & \text{si } t > t_{\text{warmup}} \end{cases} \quad (10)$$

donde T es el número total de pasos de entrenamiento.

3 Datos

3.1 Formato y estructura

El dataset utilizado consiste en un archivo JSONL (JSON Lines) donde cada línea representa un ejemplo de entrenamiento con la siguiente estructura:

```
{"instruction": "texto_instrucción",
  "input": "contexto_opcional",
  "output": "respuesta_esperada"}
```

Los campos son:

- **instruction** (obligatorio): La instrucción o pregunta que el modelo debe responder.
- **input** (opcional): Contexto adicional o datos de entrada relevantes para la instrucción.
- **output** (obligatorio): La respuesta correcta o salida esperada del modelo.

Esta estructura permite entrenar al modelo para seguir instrucciones de forma flexible, manejando tanto preguntas directas como tareas que requieren procesamiento de contexto adicional.

3.2 Características del dataset

Table 1: Resumen del dataset utilizado para entrenamiento.

Atributo	Valor
Formato	JSONL con campos <code>instruction</code> , <code>input</code> (opcional), <code>output</code>
Dominio	Instrucciones en español latinoamericano derivadas de documentos académicos y fuentes curadas
Idioma	Español (variante latinoamericana)
Tamaño aproximado	~400 MB
Tokens aproximados	~50 millones de tokens
Preparación	Limpieza, normalización y conversión a formato instruccional
Disponibilidad	Privado (no compartido por restricciones de privacidad y licencia)

3.3 Procesamiento y tokenización

El dataset se procesa mediante el tokenizador del modelo base LLaMA 3.2, que utiliza BPE (*Byte Pair Encoding*) con vocabulario de aproximadamente 128,000 tokens. Las secuencias se truncan o rellenan (*padding*) para ajustarse a la longitud máxima de 4096 tokens. Durante el entrenamiento, solo se calcula la pérdida sobre los tokens de la respuesta (`output`), ignorando la instrucción y el input para evitar que el modelo aprenda a predecir las instrucciones mismas.

3.4 Partición train/validation

El dataset se divide en conjuntos de entrenamiento (90%) y validación (10%) mediante muestreo aleatorio estratificado. La semilla aleatoria utilizada garantiza reproducibilidad de la partición. El conjunto de validación se utiliza para monitorear el *eval loss* y la perplejidad durante el entrenamiento, permitiendo detectar sobreajuste y evaluar la generalización del modelo.

Limitación de trazabilidad de datos

Por restricciones de privacidad y licencia, no se publican detalles granulares del dataset como la distribución de tipos de instrucciones, longitudes promedio de secuencias, o contenido específico de los ejemplos. Este informe se enfoca en la metodología de entrenamiento, la infraestructura utilizada y las métricas agregadas de rendimiento. Los resultados son específicos al dataset utilizado y pueden variar con otros datos.

4 Infraestructura y entorno computacional

4.1 Hardware: HPC-UCR

El entrenamiento se ejecutó en el cluster de computación de alto desempeño HPC-UCR de la Universidad de Costa Rica, específicamente en la partición de GPUs. La infraestructura disponible incluye:

Para este trabajo se utilizó un solo nodo GPU con una GPU A100 de 80GB, lo cual fue suficiente para entrenar el modelo de 1B de parámetros con QLoRA de 4 bits.

Table 2: Recursos computacionales del cluster HPC-UCR.

Recurso	Descripción	Especificación
<i>Nodos GPU</i>		
Modelo	Lenovo ThinkSystem SR670 V2	2 nodos
GPUs por nodo	NVIDIA Tensor Core A100	4 unidades de 80GB cada una
CPU por nodo	Intel Xeon Gold 6338	64 cores @ 2.0 GHz
RAM por core	—	16 GB
Memoria total GPU	—	320 GB por nodo ($4 \times 80\text{GB}$)
<i>Nodos CPU (disponibles)</i>		
Modelo	Lenovo ThinkSystem SD630 V2	16 nodos
CPUs por nodo	Intel Xeon Gold 6338	64 cores
Cores totales CPU	—	1,024 cores
RAM por core	—	16 GB

4.2 Software y dependencias

El entorno de software se configuró dentro de un entorno virtual de Python en el cluster, instalando todas las dependencias necesarias localmente para el usuario.

Table 3: Stack de software y versiones utilizadas.

Componente	Versión	Descripción/Rol
Sistema Operativo	Ubuntu 24.04 LTS	Sistema base del cluster
Python	3.8+	Lenguaje de programación
PyTorch	2.7.1	Framework de deep learning
TorchVision	0.22.1	Utilidades para visión (soporte)
TorchAudio	2.7.1	Utilidades para audio (soporte)
CUDA Toolkit	12.8	Drivers y bibliotecas para GPU
Transformers	Latest	Librería HuggingFace para LLMs
Unsloth	Latest	Optimizaciones para entrenamiento eficiente
BitsAndBytes	Latest	Cuantización y optimización de 4/8 bits
PEFT	Latest	Parameter-Efficient Fine-Tuning
Accelerate	Latest	Distribución y optimización de entrenamiento
SentencePiece	Latest	Tokenizador BPE
Datasets	Latest	Manejo de datasets HuggingFace

4.3 Gestión de trabajos con SLURM

El cluster HPC-UCR utiliza SLURM (*Simple Linux Utility for Resource Management*) como sistema de colas para gestionar la ejecución de trabajos. El script SBATCH utilizado especifica:

El entrenamiento completo se dividió en 3 bloques de 6 horas cada uno, utilizando checkpoints para reanudar automáticamente entre bloques. Esta estrategia permite adaptarse a las políticas de tiempo máximo del cluster y proporciona puntos de recuperación en caso de fallos.

Table 4: Parámetros SLURM para ejecución de entrenamiento.

Directiva	Valor	Descripción
--partition	gpu	Partición con nodos GPU
--gres	gpu:1	Solicitar 1 GPU
--mem	40G	Memoria RAM solicitada
--cpus-per-task	8	CPUs para carga de datos
--time	06:00:00	Límite de tiempo: 6 horas
--job-name	llama32_qlora	Nombre del job
--output	logs/%j.out	Archivo de salida estándar
--error	logs/%j.err	Archivo de errores

4.4 Configuración de paths y entorno

Todas las rutas críticas se configuraron de forma centralizada en el archivo `config.py` y en variables de entorno del script SLURM, evitando rutas hardcodeadas y facilitando la portabilidad entre usuarios y sistemas. Los directorios principales son:

- `$HOME/llama32_qlora`: Directorio raíz del proyecto
- `data/`: Dataset de entrenamiento (base.jsonl)
- `scripts/`: Scripts de entrenamiento e inferencia
- `outputs/`: Modelos entrenados y checkpoints
- `logs/`: Logs de ejecución de SLURM
- `models/`: Cache de modelos de HuggingFace

5 Metodología y configuración de entrenamiento

5.1 Modelo base

Se utilizó el modelo `meta-llama/Llama-3.2-1B-Instruct` de Meta AI [5] como punto de partida. Este modelo pertenece a la familia LLaMA 3.2 y cuenta con:

- **Parámetros**: 1.24 mil millones (1.24B)
- **Arquitectura**: Transformer decoder-only con atención causal
- **Contexto**: Longitud máxima de secuencia de 131,072 tokens (extendido mediante RoPE)
- **Tokenizador**: BPE con vocabulario de ~128k tokens
- **Pre-entrenamiento**: Dataset multilingüe con predominancia de inglés
- **Instruction tuning**: Afinado para seguir instrucciones mediante RLHF

La versión "Instruct" del modelo ya está adaptada para tareas de seguimiento de instrucciones, lo cual proporciona una mejor base de partida que el modelo base para nuestro objetivo de adaptación al español.

5.2 Hiperparámetros principales

La configuración completa de entrenamiento se especificó en el archivo `config.py`, permitiendo experimentación sistemática. Los hiperparámetros principales fueron:

Table 5: Hiperparámetros de entrenamiento y sus valores.

Parámetro	Valor	Justificación
<i>Entrenamiento general</i>		
Épocas (NUM_TRAIN_EPOCHS)	60	Convergencia completa observada
Pasos máximos (MAX_STEPS)	0	Usar épocas completas sin límite de pasos
Longitud máxima (MAX_SEQ_LENGTH)	4096	Balancear contexto y eficiencia
<i>Optimización</i>		
Learning rate (LEARNING_RATE)	2×10^{-4}	Estándar para fine-tuning con AdamW
Batch size (BATCH_SIZE)	2	Limitado por memoria GPU
Gradient accumulation (GRAD_ACCUM)	4	Batch efectivo de 8 secuencias
Weight decay	0.01	Regularización L2 estándar
Warmup ratio	0.03	3% de pasos iniciales para warmup
Max grad norm	1.0	Prevenir explosión de gradientes
<i>LoRA/QLoRA</i>		
Rango LoRA (LORA_R)	16	Balance capacidad/eficiencia
Alpha LoRA (LORA_ALPHA)	32	Escalado de actualizaciones
Dropout LoRA (LORA_DROPOUT)	0.05	Regularización moderada
Cuantización	4-bit (NF4)	QLoRA para eficiencia de memoria
Módulos objetivo	q-proj, v-proj, k-proj, o-proj, gate-proj, up-proj, down-proj	Matrices de atención y proyecciones finales (capas críticas para adaptación de representaciones)
<i>Evaluación y guardado</i>		
Eval strategy	steps	Evaluación cada N pasos
Eval steps (EVAL_STEPS)	200	Frecuencia de evaluación
Save strategy	steps	Guardar cada N pasos
Save steps	200	Sincronizado con evaluación
Logging steps	10	Registro frecuente de métricas
Load best at end	True	Cargar mejor checkpoint al finalizar

5.3 Justificación de hiperparámetros

Batch size y gradient accumulation: Con batch size de 2 y acumulación de 4 pasos, se logra un batch efectivo de 8 secuencias. Esto balancea el uso de memoria GPU (permitiendo secuencias de 4096 tokens) con estabilidad del gradiente.

Learning rate y scheduler: La tasa inicial de 2×10^{-4} es apropiada para fine-tuning con AdamW. El warmup durante 3% de los pasos iniciales estabiliza el entrenamiento, mientras que el decaimiento cosenoidal permite refinamiento progresivo.

Parámetros LoRA: Un rango de 16 proporciona suficiente capacidad expresiva para adaptar el modelo a español sin entrenar demasiados parámetros. El alpha de 32 ($= 2 \times r$) es una configuración estándar que ha demostrado buenos resultados.

Evaluación frecuente: Evaluar cada 200 pasos permite monitorear de cerca la convergencia y detectar tempranamente problemas como sobreajuste u oscilaciones.

5.4 Estrategia de checkpointing

Se implementó guardado automático de checkpoints cada 200 pasos de entrenamiento, sincronizado con las evaluaciones. Los checkpoints incluyen:

- Adaptadores LoRA entrenados (`adapter_model.safetensors`)
- Configuración de LoRA (`adapter_config.json`)
- Tokenizador completo (archivos `tokenizer*`)
- Estado del optimizador y scheduler
- Argumentos de entrenamiento

Esta estrategia permite:

1. Reanudar entrenamiento desde cualquier checkpoint si se interrumpe
2. Evaluar diferentes puntos del entrenamiento sin reentrenar
3. Seleccionar el mejor modelo según métricas de validación

El script SLURM detecta automáticamente si existe un checkpoint previo y reanuda desde él, lo cual fue esencial para dividir el entrenamiento en múltiples bloques de 6 horas.

6 Reproducibilidad

6.1 Semillas aleatorias y determinismo

Para garantizar reproducibilidad, se establecieron semillas aleatorias fijas en todos los componentes que introducen aleatoriedad:

- **Python random:** `random.seed(42)`
- **NumPy:** `np.random.seed(42)`
- **PyTorch CPU:** `torch.manual_seed(42)`
- **PyTorch GPU:** `torch.cuda.manual_seed_all(42)`
- **Transformers:** `set_seed(42)` de HuggingFace

Se utilizó la semilla 42 de forma consistente en todo el pipeline. Adicionalmente, se configuraron flags de determinismo en PyTorch:

```
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

Estas configuraciones garantizan reproducibilidad exacta de resultados en el mismo hardware, aunque con un pequeño costo en rendimiento.

6.2 Entorno de dependencias

El archivo `requirements.txt` especifica todas las dependencias de Python con versiones explícitas:

```
torch==2.7.1
torchvision==0.22.1
torchaudio==2.7.1
transformers==4.45.0
unsloth[colab-new]@git+https://github.com/unslothai/unsloth.git
datasets
accelerate
peft
bitsandbytes
sentencepiece
```

Para replicación exacta, se recomienda también generar un snapshot del entorno completo con:

```
pip freeze > requirements_frozen.txt
```

Este archivo captura todas las dependencias transitivas con sus versiones exactas.

6.3 Contenedorización (opcional)

Para máxima reproducibilidad cross-platform, se podría crear un contenedor Docker con:

- Sistema operativo base (Ubuntu 24.04)
- CUDA Toolkit 12.8
- Python 3.8 y dependencias congeladas
- Código fuente del proyecto

Esto permitiría ejecutar el mismo entorno exacto en diferentes clusters, aunque en este trabajo se documentó suficientemente el entorno para replicación manual.

6.4 Instrucciones de replicación

Para replicar este trabajo en otra infraestructura:

1. Clonar el repositorio: `git clone https://github.com/Alison-Lobo/llama32 qlora`
2. Ejecutar setup automático: `bash setup.sh`
3. Preparar dataset en formato JSONL y colocarlo en `data/base.jsonl`
4. Ajustar configuración en `config.py` según recursos disponibles
5. Si se usa SLURM, adaptar `scripts/train_block_full_gpu.sbatch` con las directivas apropiadas
6. Ejecutar entrenamiento: `sbatch scripts/train_block_full_gpu.sbatch` (SLURM) o `python scripts/train_llama32_gpu.py` (local)

7. Monitorear logs en tiempo real: `tail -f logs/*.out`

8. Evaluar modelo entrenado: `python scripts/infer_llama.py --model_path outputs/llama32_qlora`

Todo el código, scripts y documentación están disponibles públicamente en el repositorio [6].

7 Resultados y análisis

7.1 Métricas finales y convergencia

El entrenamiento se ejecutó durante 60 épocas completas, totalizando aproximadamente 18 horas de cómputo GPU distribuidas en 3 bloques de 6 horas cada uno. Se observó convergencia consistente en todas las métricas monitoreadas.

Table 6: Comparación de métricas iniciales vs. finales.

Métrica	Inicial	Final	Δ absoluto	Reducción (%)
Eval loss	3.08	1.70	-1.38	44.8
Perplexity	21.74	5.47	-16.27	74.8
Train loss	3.22	0.14	-3.08	95.7
Grad norm	4.09	0.29	-3.80	92.9

La reducción de 75% en perplexidad indica que el modelo mejoró sustancialmente su capacidad de modelar secuencias en español. La reducción de 96% en train loss junto con 45% en eval loss muestra aprendizaje efectivo sin sobreajuste severo (el eval loss continúa disminuyendo hasta el final).

7.2 Evolución por época

Table 7: Progreso de métricas en épocas representativas.

Época	Train loss	Eval loss	PPL	Learning rate	Grad norm
1	3.2204	3.0790	21.74	2.66×10^{-5}	4.09
10	0.2142	3.0143	20.37	1.71×10^{-4}	1.17
20	0.1775	2.9798	19.68	1.37×10^{-4}	1.07
30	0.1523	2.8126	16.65	1.03×10^{-4}	0.36
40	0.1519	2.6656	14.38	6.88×10^{-5}	0.33
50	0.1454	2.4571	11.67	3.40×10^{-5}	0.30
60	0.1392	1.7000	5.47	4.35×10^{-8}	0.29

Análisis por fases:

- **Épocas 1-10:** Descenso rápido de train loss ($3.22 \rightarrow 0.21$), con eval loss disminuyendo más lentamente ($3.08 \rightarrow 3.01$). El modelo aprende rápidamente patrones del dataset de entrenamiento. Gradient norm alto inicialmente (4.09) indica ajustes significativos.
- **Épocas 10-30:** Train loss se estabiliza en torno a 0.15-0.18, mientras eval loss continúa disminuyendo ($3.01 \rightarrow 2.81$). Esta fase muestra generalización progresiva. Gradient norm se reduce drásticamente ($1.17 \rightarrow 0.36$), indicando convergencia hacia un mínimo.

- **Épocas 30-50:** Mejora sostenida en eval loss ($2.81 \rightarrow 2.46$) y perplejidad ($16.65 \rightarrow 11.67$). El learning rate decae gradualmente, permitiendo refinamiento fino de los pesos.
- **Épocas 50-60:** Aceleración final de la convergencia, con eval loss cayendo notablemente ($2.46 \rightarrow 1.70$). Esta mejora dramática en las últimas épocas es consistente con el decaimiento cosenoidal del learning rate, que permite exploraciones muy finas del espacio de parámetros.

7.3 Comportamiento del learning rate

El scheduler de learning rate implementado muestra el comportamiento esperado:

- **Warmup (épocas 1-2):** LR aumenta linealmente desde casi cero hasta 1.71×10^{-4} (epoch 10), estabilizando el entrenamiento inicial.
- **Plateau (épocas 10-30):** LR se mantiene relativamente alto ($\sim 10^{-4}$), permitiendo exploración activa.
- **Decaimiento cosenoidal (épocas 30-60):** LR disminuye suavemente siguiendo coseno, terminando cerca de cero (4.35×10^{-8} en época 60).

Este perfil de LR permitió balance óptimo entre exploración inicial y refinamiento final.

7.4 Top-2 mejoras más significativas

Table 8: Métricas con mayor mejora porcentual.

Métrica	Inicial	Final	Reducción (%)
Train loss	3.22	0.14	95.7
Gradient norm	4.09	0.29	92.9
Perplexity	21.74	5.47	74.8

La reducción de 96% en train loss y 93% en gradient norm confirma convergencia efectiva del optimizador. La reducción de 75% en perplejidad es la métrica más relevante desde el punto de vista del modelado de lenguaje, indicando mejora sustancial en la calidad de las predicciones.

7.5 Costos computacionales

El uso de QLoRA permitió entrenar un modelo de 1.24B de parámetros utilizando menos de la mitad de la memoria de una GPU A100, dejando margen para procesar secuencias largas (4096 tokens) y mantener batch size aceptable.

7.6 Interpretación de resultados

Los resultados demuestran que:

1. **Convergencia exitosa:** Todas las métricas muestran mejora continua sin saturación o divergencia.
2. **Generalización adecuada:** El eval loss disminuye consistentemente, indicando que el modelo no simplemente memoriza sino que aprende patrones generalizables.

Table 9: Tiempos y recursos utilizados durante el entrenamiento.

Fase	Medida	Valor
Entrenamiento total	Tiempo GPU	~18 horas (3 bloques $\times$ 6 h)
	GPU utilizada	NVIDIA A100 80GB
	Uso de VRAM	~35-40 GB (picos)
	Núcleos CPU	8 cores para carga de datos
Inferencia	Tiempo por instrucción	0.8–1.2 segundos
	Modo	Generación autorecursiva con sampling
	Max tokens generados	200 por defecto

3. **Estabilidad numérica:** El gradient norm se mantiene en rangos saludables sin explosiones o desvanecimientos.
4. **Eficiencia demostrada:** 18 horas de entrenamiento en hardware académico produjo un modelo funcional, validando la viabilidad de la metodología.
5. **Viabilidad institucional:** La estrategia de entrenamiento por bloques con checkpoints automáticos permite adaptarse a limitaciones de tiempo de clusters compartidos.

Limitaciones de la evaluación

Este análisis se basa únicamente en métricas intrínsecas (loss, perplejidad) calculadas sobre un conjunto de validación del mismo dominio del entrenamiento. Para una evaluación completa se requieren benchmarks externos, evaluación humana, y pruebas en tareas downstream específicas. Los resultados son específicos al dataset utilizado y pueden no generalizarse directamente a otros dominios o variantes del español.

8 Discusión

8.1 Validación de la metodología

Este trabajo demuestra la viabilidad de adaptar modelos de lenguaje grande al español latinoamericano utilizando recursos computacionales académicos limitados. La combinación de QLoRA (reducción de memoria mediante cuantización) con estrategias de checkpoint y reanudación permite entrenar modelos que de otra forma requerirían infraestructura industrial.

La reducción sostenida de eval loss y perplejidad durante 60 épocas sin signos de sobreajuste valida la elección de hiperparámetros y la capacidad del modelo para aprender representaciones generalizables del español instructivo.

8.2 Comparación con literatura

Los resultados son consistentes con hallazgos reportados en la literatura sobre QLoRA [1], que demuestra que la cuantización de 4 bits con adaptadores LoRA puede alcanzar rendimiento comparable al fine-tuning completo con una fracción de los recursos.

La perplejidad final de 5.47 es competitiva para un modelo de 1B de parámetros adaptado a un dominio específico, aunque no directamente comparable con modelos más grandes o evaluaciones en benchmarks estándar por diferencias en datasets y métricas.

8.3 Contexto del proyecto Latam-GPT

Este trabajo se enmarca en el contexto más amplio del proyecto Latam-GPT liderado por CENIA (Chile) [7], que busca desarrollar un modelo fundacional de lenguaje para América Latina. Los objetivos específicos de esta colaboración fueron:

1. Contribuir a la recopilación y curación de datos en español costarricense para el corpus de entrenamiento.
2. Validar metodologías de entrenamiento y evaluación en infraestructura académica regional.
3. Documentar experiencias técnicas que puedan beneficiar a otras instituciones participantes.
4. Explorar la factibilidad de entrenar y evaluar modelos localmente antes de contribuir al esfuerzo centralizado.

La experiencia adquirida en este proyecto puede informar decisiones técnicas en Latam-GPT respecto a estrategias de entrenamiento distribuido, gestión de recursos compartidos, y adaptación a idiomas regionales.

8.4 Lecciones aprendidas

Técnicas:

- QLoRA es altamente efectivo para fine-tuning en GPUs con 40-80GB de VRAM.
- Checkpointing frecuente es esencial en clusters compartidos con límites de tiempo.
- Configuración centralizada (config.py) facilita experimentación sistemática.
- Warmup + cosine decay funcionó mejor que learning rate constante.

Operacionales:

- Documentación exhaustiva ahorra tiempo en troubleshooting y replicación.
- Automatización completa (setup.sh) reduce barreras de entrada para nuevos usuarios.
- Validación continua del dataset previene errores tardíos en entrenamiento.
- Logs detallados son cruciales para diagnosticar problemas en ejecuciones largas.

Institucionales:

- La infraestructura HPC-UCR es suficiente para proyectos de investigación en LLMs.
- Coordinación con administradores del cluster facilita uso eficiente de recursos.
- Compartir experiencias entre instituciones regionales multiplica el impacto.

9 Limitaciones y trabajo futuro

9.1 Limitaciones actuales

Evaluación:

- Las métricas reportadas (loss, perplejidad) son intrínsecas y no capturan completamente la calidad percibida de las respuestas.
- No se realizaron evaluaciones en benchmarks estándar de español (BETO, Spanish SQuAD, etc.).
- Falta evaluación humana sistemática de coherencia, corrección gramatical y utilidad de las respuestas.
- No se evaluó el modelo en tareas downstream específicas (clasificación, NER, QA, etc.).

Dataset:

- El dataset no es público, limitando reproducibilidad exacta de los resultados.
- No se documentaron estadísticas detalladas (distribución de tipos de instrucción, longitudes, dominios temáticos).
- Sesgo potencial hacia documentos académicos puede limitar generalización.
- Tamaño del dataset (~50M tokens) es modesto comparado con datasets industriales.

Modelo:

- LLaMA 3.2 1B es relativamente pequeño; modelos más grandes podrían dar mejores resultados.
- Solo se entrenaron adaptadores LoRA, el modelo base permanece en inglés predominantemente.
- No se exploró fine-tuning completo para comparar con QLoRA.
- No se realizó análisis de sesgos, toxicidad o safety del modelo adaptado.

Infraestructura:

- Se utilizó solo 1 GPU; entrenamiento multi-GPU podría acelerar significativamente el proceso.
- Límite de 6 horas por job requirió fragmentar el entrenamiento, añadiendo complejidad.
- No se exploró entrenamiento en GPUs de menor capacidad (V100, RTX 3090) para democratizar más el acceso.

9.2 Direcciones futuras

Evaluación externa:

- Evaluar en benchmarks estándar de español: BETO leaderboards, MLQA, XQuAD, etc.
- Implementar evaluación humana con métricas de calidad, utilidad y fluidez.
- Comparar con modelos baseline (LLaMA original, otros modelos en español).
- Medir rendimiento en tareas específicas: summarización, traducción, QA, clasificación.

Mejoras del modelo:

- Explorar modelos base más grandes (LLaMA 3.2 3B, 7B, 11B).
- Experimentar con diferentes configuraciones de LoRA (rangos mayores, módulos adicionales).
- Investigar fine-tuning completo vs. QLoRA en términos de calidad final.
- Implementar técnicas de safety: RLHF, Constitutional AI, filtros de contenido.

Datos y dominio:

- Aumentar tamaño y diversidad del dataset con fuentes variadas (web, libros, conversaciones).
- Documentar exhaustivamente el dataset (con protección de privacidad) para transparencia.
- Crear datasets de evaluación específicos para español latinoamericano.
- Explorar adaptación a dominios específicos: legal, médico, educativo, etc.

Infraestructura y eficiencia:

- Implementar entrenamiento distribuido en múltiples GPUs (DDP, FSDP).
- Explorar técnicas de compresión adicionales: pruning, distillation.
- Optimizar para inferencia: cuantización post-entrenamiento, optimización de kernels.
- Crear contenedores Docker para portabilidad cross-platform.

Colaboración regional:

- Contribuir código y experiencias al proyecto Latam-GPT de CENIA.
- Coordinar con otras universidades latinoamericanas para compartir recursos y datasets.
- Desarrollar benchmarks específicos para variantes regionales del español.
- Establecer estándares comunes para evaluación de modelos en español.

9.3 Recomendaciones metodológicas

Para futuros proyectos similares se recomienda:

1. **Planificación:** Definir claramente objetivos, métricas de éxito y recursos disponibles antes de iniciar.
2. **Dataset:** Invertir tiempo significativo en curación, limpieza y documentación de datos.
3. **Baseline:** Establecer comparaciones con modelos existentes desde el inicio.
4. **Monitoreo:** Implementar logging exhaustivo y visualización de métricas en tiempo real.
5. **Evaluación:** Planificar evaluaciones externas y humanas desde el diseño del proyecto.
6. **Documentación:** Mantener documentación actualizada durante todo el proceso, no solo al final.
7. **Reproducibilidad:** Fijar semillas, versiones y configuraciones desde el inicio.
8. **Colaboración:** Compartir experiencias y código abiertamente para beneficio de la comunidad.

10 Conclusiones

Este trabajo demostró la viabilidad de adaptar el modelo LLaMA 3.2 (1B) al español latinoamericano mediante fine-tuning eficiente con QLoRA en infraestructura académica. Los resultados principales incluyen:

- **Convergencia exitosa:** Reducción de 75% en perplejidad ($21.74 \rightarrow 5.47$) indica mejora sustancial en modelado del lenguaje.
- **Eficiencia demostrada:** QLoRA permitió entrenar en una sola GPU A100 80GB en 18 horas totales, tiempo razonable para iteración rápida.
- **Metodología reproducible:** Flujo completamente documentado con configuración centralizada, checkpointing automático y scripts de automatización.
- **Viabilidad institucional:** Se validó que universidades con clusters HPC modestos pueden participar en investigación de LLMs.
- **Contribución regional:** Experiencia técnica y código abierto disponibles para proyecto Latam-GPT y otras iniciativas similares.

Las principales limitaciones son la falta de evaluaciones externas sistemáticas y la imposibilidad de compartir el dataset por restricciones de privacidad. El trabajo futuro debe enfocarse en evaluaciones más rigurosas, expansión del dataset, exploración de modelos más grandes, y coordinación con esfuerzos regionales como Latam-GPT.

Este proyecto sienta bases sólidas para desarrollo continuo de capacidades en modelos de lenguaje para español latinoamericano dentro de instituciones académicas costarricenses, demostrando que la investigación competitiva en LLMs es alcanzable con recursos limitados mediante metodologías inteligentes y colaboración regional.

Agradecimientos

Se agradece al Centro de Investigación en Opinión y Datos (CIOdD) de la Universidad de Costa Rica por el apoyo institucional y facilitar el acceso a recursos computacionales. Especial agradecimiento al equipo de HPC-UCR por la provisión de infraestructura GPU y asistencia técnica durante la ejecución de los experimentos. Se reconoce también al proyecto Latam-GPT de CENIA (Chile) por inspirar y contextualizar este trabajo dentro de esfuerzos regionales más amplios. Finalmente, se agradece a la comunidad de código abierto, particularmente a los desarrolladores de Unsloth, HuggingFace Transformers, y PyTorch, cuyas herramientas hicieron posible este proyecto.

Declaraciones

Conflicto de intereses: Los autores declaran no tener conflictos de intereses relacionados con este trabajo.

Financiamiento: Este proyecto fue realizado utilizando infraestructura institucional de la Universidad de Costa Rica (HPC-UCR) sin financiamiento externo específico.

Ética y uso de datos: El dataset utilizado para entrenamiento fue curado siguiendo políticas institucionales de privacidad y no contiene información personal identificable. No se compartirá públicamente por restricciones de licencia y privacidad. El modelo entrenado no ha sido evaluado

exhaustivamente en términos de sesgos, toxicidad o seguridad, por lo que se recomienda precaución en su uso para aplicaciones de producción.

Disponibilidad de código: Todo el código fuente, scripts de entrenamiento, y configuración están disponibles públicamente en el repositorio: https://github.com/Alison-Lobo/llama32_qlora bajo licencia de uso académico.

References

- [1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient fine-tuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014.
- [4] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2017.
- [5] Meta AI. meta-llama/llama-3.2-1b-instruct: Model card. <https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct>. Accedido: 2025-01-08.
- [6] Alison Lobo Salas. Repositorio llama32_qlora: Fine-tuning en español con qlora. https://github.com/Alison-Lobo/llama32_qlora. Accedido: 2025-01-08.
- [7] Centro Nacional de Inteligencia Artificial - CENIA. Proyecto latam-gpt: Modelo fundacional para américa latina. <https://cenia.cl/>. Chile, 2024.

A Comandos de uso

A.1 Instalación

```
# Clonar repositorio
git clone https://github.com/Alison-Lobo/llama32_qlora
cd llama32_qlora
```

```
# Ejecutar setup automático
bash setup.sh
```

A.2 Entrenamiento local

```
# Activar entorno virtual
source venv/bin/activate

# Entrenar con configuración por defecto
python scripts/train_llama32_gpu.py

# O personalizar hiperparámetros
EPOCHS=40 LEARNING_RATE=1e-4 python scripts/train_llama32_gpu.py
```

A.3 Entrenamiento en HPC con SLURM

```
# Enviar job
cd scripts
sbatch train_block_full_gpu.sbatch

# Monitorear
squeue -u $USER
tail -f logs/llama32_qlora_full_*.out
```

A.4 Inferencia

```
# Inferencia básica
python scripts/infer_llama.py \
    --model_path outputs/llama32_qlora \
    --prompt "¿Qué es Python?"

# Modo interactivo
python scripts/infer_llama.py \
    --model_path outputs/llama32_qlora \
    --interactive
```

B Estructura de archivos del repositorio

```
llama32_qlora/
  README.md          # Documentación principal
  INSTALL.md         # Guía de instalación detallada
  requirements.txt    # Dependencias de Python
  config.py          # Configuración centralizada
  setup.sh           # Script de setup automático
  .gitignore         # Archivos ignorados por git

  scripts/           # Scripts de entrenamiento e inferencia
    train_llama32_gpu.py      # Script principal de entrenamiento
    train_block_full_gpu.sbatch # Job SLURM para HPC
    infer_llama.py           # Script de inferencia

  data/              # Datos de entrenamiento
    base.jsonl       # Dataset (no incluido)

  outputs/           # Modelos entrenados
    llama32_qlora/    # Checkpoints y adapters
      adapter_model.safetensors
      adapter_config.json
      tokenizer.json
      tokenizer_config.json
      training_summary.json
```

```
logs/                                # Logs de entrenamiento
  *.out / *.err                      # Logs de SLURM

models/                              # Cache de HuggingFace
  hf_home/
  hf_cache/
```